

Aplicar **paralelismo e otimização de performance no Playwright com Java** é essencial quando você começa a escalar testes UI.

1. Conceito rápido

No **Playwright**, paralelismo significa:

- Rodar **múltiplos testes ao mesmo tempo**
- Reduzir tempo total da suíte
- Melhor uso de CPU/memória

👉 Diferente do Selenium, o Playwright já é projetado com isolamento de contexto (BrowserContext), o que facilita paralelismo seguro.

2. Estratégias de paralelismo no Java

No Java, o Playwright **não tem runner próprio**, então usamos:

- JUnit 5 (recomendado)
- TestNG (alternativa)

3. Paralelismo com JUnit 5

3.1 Habilitando paralelismo

Crie o arquivo:

```
src/test/resources/junit-platform.properties
```

Conteúdo:

```
junit.jupiter.execution.parallel.enabled = true  
junit.jupiter.execution.parallel.mode.default = concurrent  
junit.jupiter.execution.parallel.config.strategy = dynamic
```

3.2 Exemplo de teste paralelo

```
import com.microsoft.playwright.*;  
  
import org.junit.jupiter.api.Test;
```

```

public class TesteParalelo {

    @Test
    void teste1() {
        executarTeste("https://example.com");
    }

    @Test
    void teste2() {
        executarTeste("https://example.org");
    }

    void executarTeste(String url) {
        try (Playwright playwright = Playwright.create()) {
            Browser browser = playwright.chromium().launch(
                new BrowserType.LaunchOptions().setHeadless(true)
            );

            BrowserContext context = browser.newContext();
            Page page = context.newPage();

            page.navigate(url);
            System.out.println("Título: " + page.title());
            browser.close();
        }
    }
}

```

- ✓ Cada teste roda isolado
- ✓ Pode executar simultaneamente

⚡ 4. Melhorando performance (BOAS PRÁTICAS)

Aqui é onde muita gente erra — criar browser toda hora é caro.

✗ Errado (lento)

- Abrir browser em cada teste

✓ Correto (rápido)

- Reutilizar browser
- Criar contextos isolados

🔗 4.1 Reutilizando Browser (melhor abordagem)

```
import com.microsoft.playwright.*;

import org.junit.jupiter.api.*;

@TestInstance(TestInstance.Lifecycle.PER_CLASS)

public class TestePerformance {

    Playwright playwright;

    Browser browser;

    @BeforeAll

    void setup() {

        playwright = Playwright.create();

        browser = playwright.chromium().launch(

            new BrowserType.LaunchOptions().setHeadless(true)

        );

    }

    @AfterAll

    void teardown() {

        browser.close();

    }

}
```

```
    playwright.close();  
}
```

```
@Test  
void teste1() {  
    executarTeste("https://example.com");  
}
```

```
@Test  
void teste2() {  
    executarTeste("https://example.org");  
}
```

```
void executarTeste(String url) {  
    BrowserContext context = browser.newContext();  
    Page page = context.newPage();  
  
    page.navigate(url);  
    System.out.println(page.title());  
  
    context.close(); // importante!  
}  
}
```

🔥 Ganho enorme de performance

🔥 Seguro para paralelismo

7. Dicas reais de performance (muito importantes)

✅ Use headless

```
.setHeadless(true)
```

✅ Evite sleep

Use auto-wait do Playwright

✓ **Reutilize browser**

→ maior ganho de performance

✓ **Use context isolado**

→ evita conflitos

✓ **Reduza recursos carregados**

⚠ **8. Armadilhas comuns**

- ✗ Compartilhar Page entre threads
- ✗ Não fechar context
- ✗ Criar browser por teste
- ✗ Usar sleep ao invés de auto-wait

🎯 **9. Arquitetura recomendada**

1. Classe Base (Browser único)
2. Testes paralelos
3. Context por teste
4. Page Object Model (POM)

💡 **10. Exemplo com POM + paralelismo**

```
public class LoginPage {  
  
    private Page page;  
  
    public LoginPage(Page page) {  
        this.page = page;  
    }  
  
    public void acessar() {  
        page.navigate("https://site.com/login");  
    }  
}
```

```
}  
}
```

Classe teste:

```
@Test  
  
void testeLogin() {  
  
    BrowserContext context = browser.newContext();  
  
    Page page = context.newPage();  
  
    LoginPage login = new LoginPage(page);  
  
    login.acessar();  
  
    context.close();  
  
}
```

11. Classe Base (responsável pelo Browser)

Aqui está o ponto chave:

```
import com.microsoft.playwright.*;  
import org.junit.jupiter.api.*;  
  
@TestInstance(TestInstance.Lifecycle.PER_CLASS)  
public class BaseTest {  
  
    protected Playwright playwright;  
    protected Browser browser;  
  
    @BeforeAll  
    void setup() {  
  
        playwright = Playwright.create();  
  
        browser = playwright.chromium().launch(  
            new BrowserType.LaunchOptions().setHeadless(true)  
        );  
    }  
}
```

```
}
```

```
@AfterAll
```

```
void teardown() {
```

```
    browser.close();
```

```
    playwright.close();
```

```
}
```

```
}
```

👉 Aqui você cria **uma única instância de browser**

👉 Isso melhora MUITO a performance

12. Classe de Teste usando a Base

```
import com.microsoft.playwright.*;
```

```
import org.junit.jupiter.api.Test;
```

```
public class LoginTest extends BaseTest {
```

```
@Test
```

```
void teste1() {
```

```
    BrowserContext context = browser.newContext();
```

```
    Page page = context.newPage();
```

```
    page.navigate("https://example.com");
```

```
    System.out.println(page.title());
```

```
    context.close();
```

```
}
```

```
@Test
```

```
void teste2() {  
  
    BrowserContext context = browser.newContext();  
  
    Page page = context.newPage();  
  
    page.navigate("https://example.org");  
  
    System.out.println(page.title());  
  
    context.close();  
  
}  
}
```

🔥 13. Por que isso funciona bem com paralelismo?

Porque o Playwright foi projetado assim:

- ✓ Browser → compartilhado
- ✓ BrowserContext → isolado (thread-safe)
- ✓ Page → isolado por teste

⚠️ 14. Erro comum

❌ NÃO faça isso:

```
Page page = browser.newPage(); // errado em paralelo
```

👉 Isso quebra isolamento

✓ Sempre:

```
BrowserContext context = browser.newContext();  
  
Page page = context.newPage();
```